

Travel Itinerary Management System: Backend Architecture and Implementation

This comprehensive report outlines the development of a backend system for managing travel itineraries, focusing on Thailand's popular destinations Phuket and Krabi. The system includes database architecture using SQLAlchemy, RESTful API endpoints with FastAPI, and an MCP server for recommending itineraries based on trip duration.

Database Architecture

The database architecture is designed to efficiently store and manage travel itineraries with proper relationships between various entities. Instead of grouping everything under a generic "ITEM" category, we'll create specialized tables for each conceptually different entity, following best practices for relational database design^[1].

Core Models

Trip Model

```
from sqlalchemy import Column, Integer, String, DateTime, ForeignKey, Float, Table
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy.orm import relationship
import datetime

Base = declarative_base()

class Trip(Base):
    __tablename__ = "trips"

    id = Column(Integer, primary_key=True, index=True)
    name = Column(String, nullable=False)
    description = Column(String)
    duration_nights = Column(Integer, nullable=False)
    created_at = Column(DateTime, default=datetime.datetime.utcnow)
    updated_at = Column(DateTime, default=datetime.datetime.utcnow, onupdate=datetime.datetime.utcnow)
    user_id = Column(Integer, ForeignKey("users.id"))
    is_recommended = Column(Boolean, default=False)

    # Relationships
    user = relationship("User", back_populates="trips")
    accommodations = relationship("Accommodation", back_populates="trip", cascade="all, delete-orphan")
    transfers = relationship("Transfer", back_populates="trip", cascade="all, delete-orphan")
    activities = relationship("Activity", back_populates="trip", cascade="all, delete-orphan")
    itinerary_days = relationship("ItineraryDay", back_populates="trip", cascade="all, delete-orphan")
```

Location Model

```
class Location(Base):
    __tablename__ = "locations"

    id = Column(Integer, primary_key=True, index=True)
    name = Column(String, nullable=False)
    region = Column(String, nullable=False) # e.g., "Phuket", "Krabi"
    description = Column(String)
    latitude = Column(Float)
    longitude = Column(Float)

    # Relationships
    hotels = relationship("Hotel", back_populates="location")
    attractions = relationship("Attraction", back_populates="location")
```

Hotel Model

```
class Hotel(Base):
    __tablename__ = "hotels"

    id = Column(Integer, primary_key=True, index=True)
    name = Column(String, nullable=False)
    location_id = Column(Integer, ForeignKey("locations.id"), nullable=False)
    description = Column(String)
    rating = Column(Float)
    price_per_night = Column(Float)
    amenities = Column(String)
    image_url = Column(String)

    # Relationships
    location = relationship("Location", back_populates="hotels")
    accommodations = relationship("Accommodation", back_populates="hotel")
```

Accommodation Model

```
class Accommodation(Base):
    __tablename__ = "accommodations"

    id = Column(Integer, primary_key=True, index=True)
    trip_id = Column(Integer, ForeignKey("trips.id"), nullable=False)
    hotel_id = Column(Integer, ForeignKey("hotels.id"), nullable=False)
    check_in_date = Column(DateTime, nullable=False)
    check_out_date = Column(DateTime, nullable=False)
    room_type = Column(String)
    price = Column(Float)

    # Relationships
    trip = relationship("Trip", back_populates="accommodations")
    hotel = relationship("Hotel", back_populates="accommodations")
```

Attraction and Activity Models

```
class Attraction(Base):
    __tablename__ = "attractions"

    id = Column(Integer, primary_key=True, index=True)
    name = Column(String, nullable=False)
    location_id = Column(Integer, ForeignKey("locations.id"), nullable=False)
    description = Column(String)
    category = Column(String) # e.g., "Beach", "Temple", "Adventure"
    duration_hours = Column(Float)
    price = Column(Float)

    # Relationships
    location = relationship("Location", back_populates="attractions")
    activities = relationship("Activity", back_populates="attraction")

class Activity(Base):
    __tablename__ = "activities"

    id = Column(Integer, primary_key=True, index=True)
    trip_id = Column(Integer, ForeignKey("trips.id"), nullable=False)
    attraction_id = Column(Integer, ForeignKey("attractions.id"), nullable=False)
    day_number = Column(Integer, nullable=False)
    start_time = Column(DateTime)
    end_time = Column(DateTime)
    notes = Column(String)

    # Relationships
    trip = relationship("Trip", back_populates="activities")
    attraction = relationship("Attraction", back_populates="activities")
    itinerary_day = relationship("ItineraryDay", back_populates="activities")
```

Transfer Model

```
class Transfer(Base):
    __tablename__ = "transfers"

    id = Column(Integer, primary_key=True, index=True)
    trip_id = Column(Integer, ForeignKey("trips.id"), nullable=False)
    from_location_id = Column(Integer, ForeignKey("locations.id"), nullable=False)
    to_location_id = Column(Integer, ForeignKey("locations.id"), nullable=False)
    transfer_date = Column(DateTime, nullable=False)
    transfer_type = Column(String) # e.g., "Taxi", "Ferry", "Flight"
    price = Column(Float)

    # Relationships
    trip = relationship("Trip", back_populates="transfers")
    from_location = relationship("Location", foreign_keys=[from_location_id])
    to_location = relationship("Location", foreign_keys=[to_location_id])
    itinerary_day = relationship("ItineraryDay", back_populates="transfers")
```

ItineraryDay Model

```
class ItineraryDay(Base):
    __tablename__ = "itinerary_days"

    id = Column(Integer, primary_key=True, index=True)
    trip_id = Column(Integer, ForeignKey("trips.id"), nullable=False)
    day_number = Column(Integer, nullable=False)
    date = Column(DateTime)
    description = Column(String)

    # Relationships
    trip = relationship("Trip", back_populates="itinerary_days")
    activities = relationship("Activity", back_populates="itinerary_day")
    transfers = relationship("Transfer", back_populates="itinerary_day")
```

Database Seeding

To seed the database with realistic data for Phuket and Krabi, we'll utilize the information from the search results:

1. **Locations:** Add Phuket and Krabi as main regions, along with key areas like Patong, Kata Beach, Karon Beach, Ao Nang, and Railay Beach.
2. **Hotels:** Populate with properties from both regions using the data provided:
 - Phuket hotels: Amari Phuket, Phuket Marriott Resort & Spa, Thavorn Palm Beach Resort, Hyatt Regency Phuket Resort, etc. ^[2]
 - Krabi hotels: Holiday Inn Resort Krabi, Avani Ao Nang Cliff Resort, Panan Krabi Resort, Krabi Tipa Resort, etc. ^[3]
3. **Attractions:** Add popular activities and sights in both regions:
 - Phuket attractions: Phi Phi Islands, Similan Islands, Big Buddha, James Bond Island, etc. ^[4]
 - Krabi attractions: Railay Beaches & Caves, Thung Teao Forest Natural Park, Wat Tham Sua, Hot Springs, etc. ^[5]
4. **Recommended Itineraries:** Create sample itineraries of various durations (2-8 nights) that showcase the best experiences of each region.

RESTful API Implementation with FastAPI

FastAPI provides a modern framework for building APIs with automatic validation, serialization, and documentation. We'll implement the required endpoints as follows:

1. Models and Schemas

First, define Pydantic models for request and response validation:

```
from pydantic import BaseModel, Field
from typing import List, Optional
from datetime import datetime

class HotelBase(BaseModel):
    name: str
    rating: Optional[float] = None
    price_per_night: float

class HotelCreate(HotelBase):
    location_id: int
    description: Optional[str] = None
    amenities: Optional[str] = None
    image_url: Optional[str] = None

class HotelResponse(HotelBase):
    id: int
    location_id: int

    class Config:
        orm_mode = True

# Similar schemas for other models...

class TripCreate(BaseModel):
    name: str
    description: Optional[str] = None
    duration_nights: int
    user_id: Optional[int] = None

class TripResponse(BaseModel):
    id: int
    name: str
    description: Optional[str] = None
    duration_nights: int
    created_at: datetime
    updated_at: datetime

    class Config:
        orm_mode = True

class TripDetailResponse(TripResponse):
    accommodations: List[AccommodationResponse]
    activities: List[ActivityResponse]
    transfers: List[TransferResponse]

    class Config:
        orm_mode = True
```

2. API Endpoints

Implement the required endpoints for creating and viewing itineraries:

```
from fastapi import FastAPI, Depends, HTTPException, status
from sqlalchemy.orm import Session
from typing import List

from . import models, schemas, crud, database

app = FastAPI(title="Travel Itinerary API")

# Dependency
def get_db():
    db = database.SessionLocal()
    try:
        yield db
    finally:
        db.close()

@app.post("/trips/", response_model=schemas.TripResponse, status_code=status.HTTP_201_CREATED)
def create_trip(trip: schemas.TripCreate, db: Session = Depends(get_db)):
    """
    Create a new trip itinerary
    """
    return crud.create_trip(db=db, trip=trip)

@app.get("/trips/", response_model=List[schemas.TripResponse])
def read_trips(skip: int = 0, limit: int = 100, db: Session = Depends(get_db)):
    """
    Retrieve all trip itineraries
    """
    trips = crud.get_trips(db, skip=skip, limit=limit)
    return trips

@app.get("/trips/{trip_id}", response_model=schemas.TripDetailResponse)
def read_trip(trip_id: int, db: Session = Depends(get_db)):
    """
    Retrieve a specific trip itinerary by ID
    """
    db_trip = crud.get_trip(db, trip_id=trip_id)
    if db_trip is None:
        raise HTTPException(status_code=404, detail="Trip not found")
    return db_trip

@app.post("/trips/{trip_id}/activities/", response_model=schemas.ActivityResponse)
def add_activity(
    trip_id: int, activity: schemas.ActivityCreate, db: Session = Depends(get_db)
):
    """
    Add an activity to a trip itinerary
    """
    return crud.create_activity(db=db, activity=activity, trip_id=trip_id)

# Similar endpoints for accommodations and transfers...
```

3. CRUD Implementation

Create the CRUD operations to handle database interactions^[6]:

```
def create_trip(db: Session, trip: schemas.TripCreate):
    db_trip = models.Trip(
        name=trip.name,
        description=trip.description,
        duration_nights=trip.duration_nights,
        user_id=trip.user_id
    )
    db.add(db_trip)
    db.commit()
    db.refresh(db_trip)
    return db_trip

def get_trip(db: Session, trip_id: int):
    return db.query(models.Trip).filter(models.Trip.id == trip_id).first()

def get_trips(db: Session, skip: int = 0, limit: int = 100):
    return db.query(models.Trip).offset(skip).limit(limit).all()

# Similarly implement CRUD functions for other entities...
```

MCP Server for Itinerary Recommendations

The MCP (Master Control Program) server will provide itinerary recommendations based on the requested duration:

```
from fastapi import APIRouter, Depends, HTTPException
from sqlalchemy.orm import Session
from typing import List

from . import models, schemas, database

router = APIRouter(prefix="/recommendations", tags=["recommendations"])

@router.get("/", response_model=List[schemas.TripResponse])
def get_recommended_itineraries(nights: int, db: Session = Depends(database.get_db)):
    """
    Get recommended itineraries based on the requested number of nights
    """
    if nights < 2 or nights > 8:
        raise HTTPException(
            status_code=400,
            detail="Recommended itineraries are available for 2-8 nights only"
        )

    # Query the database for recommended itineraries matching the requested duration
    recommended_trips = db.query(models.Trip).filter(
        models.Trip.is_recommended == True,
        models.Trip.duration_nights == nights
    ).all()
```

```

    if not recommended_trips:
        raise HTTPException(
            status_code=404,
            detail=f"No recommended itineraries found for {nights} nights"
        )

    return recommended_trips

@router.get("/phuket", response_model=List[schemas.TripResponse])
def get_phuket_recommendations(nights: int, db: Session = Depends(database.get_db)):
    """
    Get Phuket-specific recommended itineraries
    """
    # Implementation would query for Phuket recommendations
    # For a 2-day Phuket itinerary, we could use the sample from reference[^7]
    pass

@router.get("/krabi", response_model=List[schemas.TripResponse])
def get_krabi_recommendations(nights: int, db: Session = Depends(database.get_db)):
    """
    Get Krabi-specific recommended itineraries
    """
    # Implementation would query for Krabi recommendations
    pass

```

Sample Recommended Itineraries

For a 2-night trip to Phuket, we could follow this itinerary structure based on search result [\[7\]](#):

```

def seed_phuket_itineraries(db: Session):
    # 2-day itinerary
    two_day_trip = models.Trip(
        name="Phuket Explorer: 2 Days of Paradise",
        description="A perfect 2-day introduction to Phuket's highlights, combining spirit",
        duration_nights=2,
        is_recommended=True
    )
    db.add(two_day_trip)
    db.commit()

    # Add day 1 itinerary
    day1 = models.ItineraryDay(
        trip_id=two_day_trip.id,
        day_number=1,
        description="Explore Phuket's cultural landmarks and southern beaches."
    )
    db.add(day1)

    # Add day 1 activities
    activities_day1 = [
        models.Activity(
            trip_id=two_day_trip.id,
            day_number=1,
            attraction_id=1, # Promthep Cape for sunrise
            start_time="06:00",

```



```

        end_time="07:30",
        notes="Watch the beautiful sunrise"
    ),
    models.Activity(
        trip_id=two_day_trip.id,
        day_number=1,
        attraction_id=2, # Big Buddha
        start_time="08:00",
        end_time="10:00",
        notes="Visit the iconic 140-foot Buddha statue"
    ),
    models.Activity(
        trip_id=two_day_trip.id,
        day_number=1,
        attraction_id=3, # Wat Chalong
        start_time="10:30",
        end_time="12:00",
        notes="Explore Phuket's most important Buddhist temple"
    ),
    models.Activity(
        trip_id=two_day_trip.id,
        day_number=1,
        attraction_id=4, # Kata Beach
        start_time="13:00",
        end_time="17:00",
        notes="Relax on the beach, enjoy water sports and beachfront dining"
    )
]

db.add_all(activities_day1)
db.commit()

# Similar implementation for day 2 and other durations...

```

Database Setup and Configuration

The complete database setup involves creating connections, session management, and proper configuration:

```

from sqlalchemy import create_engine
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy.orm import sessionmaker

SQLALCHEMY_DATABASE_URL = "postgresql://username:password@localhost:5432/itinerary_db"

engine = create_engine(SQLALCHEMY_DATABASE_URL)
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)

Base = declarative_base()

def init_db():
    Base.metadata.create_all(bind=engine)

```

FastAPI Application Configuration

The main application file brings everything together:

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware

from .database import init_db
from .routers import trips, activities, accommodations, transfers, recommendations

app = FastAPI(
    title="Travel Itinerary Management System",
    description="API for managing travel itineraries for Phuket and Krabi",
    version="1.0.0"
)

# CORS middleware setup
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

# Initialize database
@app.on_event("startup")
def startup_db_client():
    init_db()

# Include routers
app.include_router(trips.router)
app.include_router(activities.router)
app.include_router(accommodations.router)
app.include_router(transfers.router)
app.include_router(recommendations.router)

@app.get("/")
def read_root():
    return {"message": "Welcome to Travel Itinerary API"}
```

Conclusion

This backend system provides a comprehensive solution for managing travel itineraries, with a focus on Phuket and Krabi destinations in Thailand. The system includes:

1. A well-designed database architecture with SQLAlchemy models that capture all the essential elements of travel itineraries, including accommodations, activities, and transfers.
2. RESTful API endpoints built with FastAPI that enable creating and viewing itineraries with proper validation and error handling.
3. An MCP server component that recommends itineraries based on the requested duration, utilizing pre-seeded recommended trips.

The implementation follows best practices for database design, API development, and code organization. The system is designed to be scalable and maintainable, with clear separation of concerns and modular components.

For further enhancements, you could consider adding user authentication, implementing more sophisticated recommendation algorithms, or extending the system to support additional destinations beyond Phuket and Krabi.



1. <https://stackoverflow.com/questions/27644540/database-schema-for-itinerary>
2. <https://www.makemytrip.com/hotels-international/thailand/phuket-hotels/>
3. <https://www.makemytrip.com/hotels-international/thailand/krabi-hotels/>
4. <https://www.thrillophilia.com/things-to-do-in-phuket>
5. <https://www.thrillophilia.com/things-to-do-in-krabi>
6. <https://apidog.com/blog/how-to-quickly-implement-crud-operations-with-fastapi/>
7. <https://www.viator.com/en-IN/blog/Phuket/d349/How-to-Spend-2-Days-in-Phuket/i26687>